Cryptography Notes

1 Encryption Scheme

An **encryption scheme** is a formal way to send message securely. It conists of three sets:

1. **Plaintexts** M, the set of possible messages
2. **Ciphertexts** C, the set of encrypted messages
3. **Keys** K, the set of secret keys, i.e. a piece of information only the sender and receiver knows that is used to encrypt and decript messages

It also includes three algorithms:

1. A **key generator** `Gen()`, which returns a uniformly sampled secret key $k \in K$
2. A **encryptor** `Enc()`, which takes inputs of a secret key $k \in K$ and plaintext $m \in M$ and returns a ciphertext c
3. A **decryptor** `Dec()`, which takes inputs of the same secret key $k \in K$ and ciphertext c, and returns the plaintext m

Properties:

- An encryption scheme is **correct** iff $\forall m \in M, k \in K$: `Dec(`k, `Enc(`k, m`))` $= m$.
- An encryption scheme is **perfectly indistinguishable** if for any two messages $m_0, m_1 \in M$, the probability that a ciphertext c came from encrypting m_0 is equal to the probability that c came from encrypting m_1. More formally:

$$\Pr_{k=\texttt{Gen()}}[\texttt{Enc}(k, m_0) = c] = \Pr_{k=\texttt{Gen()}}[\texttt{Enc}(k, m_1) = c].$$

  A perfectly indistinguishable encryption scheme must have $|K| \geq |M|$, i.e. at least as many secret keys as messages.

2 Shannon's One-Time Pad

The encryption scheme for Shannon's One-Time Pad is defined as follows:

- $M = C = K = \{0,1\}^n$: The plaintexts, ciphertexts, and keys are n-bit strings
- `Gen()` returns a uniformly sampled secret key $k \in \{0,1\}^n$
- `Enc(`k, m`)` returns the ciphertext $c = k \oplus m$
- `Dec(`k, c`)` returns the plaintext $m = k \oplus c$

Properties:

- **Correctness**: Clearly, `Dec(`k, `Enc(`k, m`))` $= k \oplus (k \oplus m) = m$
- **Perfectly Indistinguishable**: We want to show that:

$$\Pr_{k=\texttt{Gen()}}[k \oplus m_0 = c] = \Pr_{k=\texttt{Gen()}}[k \oplus m_1 = c].$$

  Rearranging the equation in the LHS, we XOR both sides with m_0 and get $Pr[k = c \oplus m_0]$. Since `Gen()` returns a uniformly sampled $k \in K$, it must be that $Pr[k = \text{any } n\text{-bit string}] = 2^{-n}$. Thus, both probabilities above are equal to 2^{-n}.

Notice that because XOR cancels, the One-Time Pad is only secure if each $k \in K$ is used exactly once. For instance, suppose we have two cyphertexts $c_0 = k \oplus m_0$ and $c_1 = k \oplus m_1$. We can easily find the secret key K by solving $c_0 \oplus c_1 = (k \oplus m_0) \oplus (k \oplus m_1) = m_0 \oplus m_1$, and subsequently decrypt m_0 and m_1.

3 Modular Exponentiation

Modulo: Given positive integers g and q,

$$g \bmod q = \text{remainder when dividing } g \text{ by } q \in \{0, 1, \ldots, q-1\}.$$

Modular Exponentiation: Given positive integers g, q, and a, we compute

$$g^a \bmod q \in \{0, 1, \ldots, q-1\}.$$

The **order** of $g \bmod q$ is the smallest integer $a > 0$ such that $g^a \bmod q = 1$.

3.1 Repeated Squaring Algorithm

Suppose that g, a, and q are all n-bit integers. The naive way to compute $g^a \bmod q$ is to multiply $g \cdot g \cdot \ldots$ for a times. This takes $O(a) = O(2^n)$ multiplications, which is highly inefficient especially if a is large.

Instead, we can use the **Repeated Squaring Algorithm** as summarized below:

Algorithm 1 RepeatedSquaring(g, a, q)

```
a ← b_{n-1}b_{n-2}...b_0 (write a in binary)
A ← 1
x ← g mod q
for i = 0 to n − 1 do
    if b_i = 1 then
        A ← (A · x) mod q
    end if
    x ← x^2 mod q
end for
return A
```

Why does this algorithm work? First, we express a in binary:

$$a = b_0 + 2b_1 + 2^2 b_2 + \cdots + 2^{n-1} b_{n-1}, \text{ with each } b_i \in \{0, 1\}.$$

Then, we can rewrite g^a as:

$$\begin{aligned} g^a &= g^{b_0 + 2b_1 + 2^2 b_2 + \cdots + 2^{n-1} b_{n-1}} \\ &= g^{b_0} \cdot g^{2b_1} \cdot g^{2^2 b_2} \cdot \ldots \cdot g^{2^{n-1} b_{n-1}} \\ &= (g^1)^{b_0} \cdot (g^2)^{b_1} \cdot (g^{2^2})^{b_2} \cdot \ldots \cdot (g^{2^{n-1}})^{b_{n-1}} \\ &= \prod_{i=0}^{n-1} (g^{2^i})^{b_i}. \end{aligned}$$

Now, all we need to do is to solve for each g^{2^i} efficiently. To do this, we define the sequence h_i as so:

$$\begin{aligned} h_0 &= g \bmod q \\ h_i &= (h_{i-1})^2 \bmod q \qquad \text{for } i \geq 1. \end{aligned}$$

Note that in general, $h_i = g^{2^i} \bmod q$. Finally, using the following property of modulo:

$$(x \cdot y) \bmod q = ((x \bmod q)(y \bmod q)) \bmod q,$$

we can rewrite $g^a \bmod q$ as:

$$\begin{aligned} g^a \bmod q &= \prod_{i=0}^{n-1} (g^{2^i})^{b_i} \bmod q \\ &= \prod_{i=0}^{n-1} ((g^{2^i}) \bmod q)^{b_i} \bmod q \\ &= \prod_{i=0}^{n-1} (h_i)^{b_i} \bmod q. \end{aligned}$$

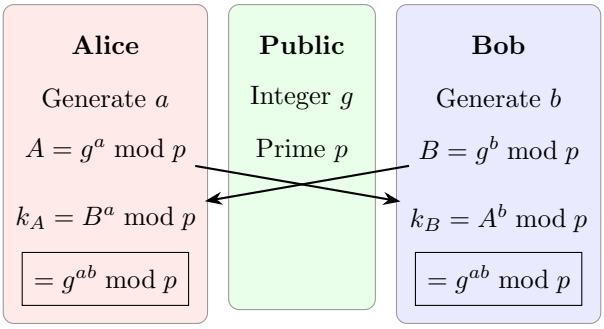


Figure 1: Diagram of the Diffie-Hellman Key Exchange.

Computing the sequence $\{h_i\}$ costs $O(n)$ multiplications (since for each h_i, you simply need to multiply $(h_{i-1})(h_{i-1})$). The product of all $(h_i)^{b_i} \bmod q$ also takes $O(n)$ multiplications, so in total we only need $O(\log a) = O(n)$ multiplications, which is significantly better than the naive method.

Note: This is not the overall running time of the Repeated Squaring Algorithm. Assuming that the multiplication of two n-bit numbers costs $O(n \log n)$ and the division of two such numbers costs $O(n^2)$:

- Computing each $(h_i)^{b_i} \bmod q$ costs $O(n^2)$, multiplying this to the current product costs $O(n \log n)$, so the cost per iteration is $O(n^2 + n \log n) = O(n^2)$.
- As stated above, the total amount of iterations is $O(n)$.

Thus, the overall running time of this algorithm is $O(n^2) \cdot O(n) = O(n^3) = O((\log a)^3)$.

3.2 Discrete Logarithm

Discrete Logarithm: The inverse of modular exponentiation. More formally, given positive integers g, q, and A, find the integer a such that:

$$A = g^a \bmod q.$$

There is no known efficient algorithm that solves this problem.

4 Key Exchange/Key Agreement

4.1 Definitions

A **key exchange/key agreement** is a protocol where two parties (*Alice* and *Bob*) securely exchange messages over a public channel. Each party independently outputs a secret key k_A and k_B such that $k_A = k_B$, and an adversary observing this communication cannot obtain the key.

A **secret** is a randomly chosen value that is kept private by the party that generated it and used to compute the shared key.

4.2 Diffie-Hellman Key Exchange

The **Diffie-Hellman Key Exchange** (Figure 1) assumes that the discrete logarithm is hard to find:

1. Let g be an integer and p be a large prime. g and p are both **public** (i.e. visible to everyone).
2. *Alice* samples a secret a and computes $A = g^a \bmod p$. She sends A to *Bob*.
3. *Bob* samples a secret b and computes $B = g^b \bmod p$. He sends B to *Alice*.
4. *Alice* computes the key:
$$k_A = B^a \bmod p = (g^b \bmod p)^a \bmod p = g^{ab} \bmod p.$$
5. *Bob* also computes the key:
$$k_B = A^b \bmod p = (g^a \bmod p)^b \bmod p = g^{ab} \bmod p.$$

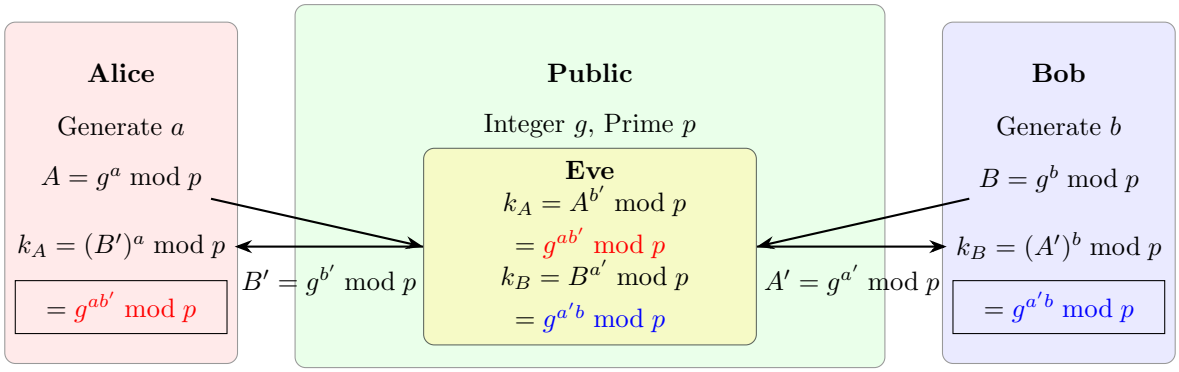


Figure 2: Diagram of the "Man in the Middle" attack.

4.3 Limitations of Diffie-Hellman

Man in the Middle Attack (Figure 2). Diffie-Hellman assumes that nobody in the public channel intercepts *Alice* and *Bob*'s message. After the public values g and p and secrets a and b are generated, suppose the following scenario occurs:

1. *Eve* intercepts *Alice*'s public value $A = g^a \bmod p$ and prevents it from reaching *Bob*. Instead, *Eve* sends *Bob* her own public value, $A' = g^{a'} \bmod p$. Similarly, she also intercepts *Bob*'s public value $B = g^b \bmod p$ and sends *Alice* another public value, $B' = g^{b'} \bmod p$.

2. Neither *Alice* nor *Bob* knows that *Eve* has changed the original messages. They will calculate their keys as normal:

$$k_A = (B')^a \bmod p = g^{ab'} \bmod p$$
$$k_B = (A')^b \bmod p = g^{a'b} \bmod p$$

3. *Eve* now knows these values: her own secret a', her public values A' and B', and *Alice* and *Bob*'s public values A and B. She can find the shared keys as follows:

$$k_A = A^{b'} \bmod p = g^{ab'} \bmod p$$
$$k_B = B^{a'} \bmod p = g^{a'b} \bmod p$$

4. Now, *Eve* shares k_A with *Alice* and k_B with *Bob*. So, when *Alice* sends her encrypted message $\texttt{Enc}(k_A, m)$, *Eve* can decrypt *Alice*'s message with $\texttt{Dec}(k_A, m)$ and re-encrypt the message with $\texttt{Enc}(k_B, m)$ to send to *Bob*. (The analogous logic holds for the reverse direction).

Side-Channel Attack. Recall that we use the Repeated Squaring Algorithm to efficiently compute A. The computer executing this algorithm behaves as follows:

- If the bit is 0, the computer only does one operation (square).
- If the bit is 1, the computer does two operations (square and multiply). This extra multiplication will take slightly more time and electricity.

Even though an interceptor *Eve* may not be able to find out a from A alone (as finding the discrete logarithm is assumed to be computationally hard), she can instead measure (1) how long each step takes and (2) how much electricity the computer uses during each step. From these measurements, she can distinguish between 0 and 1 bits. In particular, if a has n bits and *Eve* knows that a has k 1 bits, this significantly reduces her search space from 2^n to $\binom{n}{k}$.

Predictability of a. The security of Diffie-Hellman also relies on the assumption that a is random and unpredictable. If there are only few possible values of a, then *Eve* can simply compute $g^a \bmod q$ for each candidate value (as finding the modular exponent is efficient) then compare the result with A.

5 Greatest Common Divisor (GCD)

Given two integers n and m, their **greatest common divisor** (GCD) is the largest integer d that divides both n and m without a remainder.

In many cryptographic protocols, we use modular arithmetic on extremely large numbers, so it is important to understand which operations can be done efficiently in this setting. Computing the GCD is one such operation.

5.1 Euclid's Algorithm

This recursively computes the GCD of two n-bit numbers in $O(n^3)$ time (the proof is omitted, but the key fact is that within two iterations, the larger of the two numbers decreases by at least a factor of two, which bounds the total number of iterations to $O(n)$):

Algorithm 2 GCD(N, M) (assuming $N \geq M$)

1: $x \leftarrow \lfloor \frac{N}{M} \rfloor$
2: $R \leftarrow N - Mx$ (the remainder of $\frac{N}{M}$)
3: **if** $R = 0$ **then**
4: **return** M
5: **else**
6: **return** GCD(M, R)
7: **end if**